

PROMPT ENGINEERING

Customer Support Chatbot Scenario

A Comparative Study of Basic, Structured, and Robust System Prompts

Case Study: NovaSoft Project-Management Software

Compiled by

Kaneez Fatima

1. Objective

I conducted an experiment to examine how different system prompts affect the behavior of an AI customer-support chatbot.

The experiment used a fictional software company, NovaSoft, which provides project-management software for businesses. The chatbot was expected to answer common customer questions while avoiding unsupported claims about company policies, customer accounts, billing information, and future product features.

I created three progressively different system prompts and tested them using the same set of customer questions. The purpose was to observe how adding structure, constraints, and examples changed the model's responses.

2. Experimental Setup

The chatbot was tested using the following scenario:

NovaSoft provides project-management software for businesses. The chatbot helps customers with common support questions, billing issues, account problems, and general product questions.

An important limitation was established:

The model does not have access to NovaSoft's private systems or real customer accounts.

The experiment was implemented in Python using direct API calls to the Gemini API. No prompt-engineering frameworks were used.

The model used was Gemini 3.5 Flash.

The same six test cases were used across the three prompts wherever API availability allowed.

3. System Prompts

Prompt 1: Basic

The first prompt was intentionally simple:

You are a customer support agent for NovaSoft, a software company. Help customers with their questions and problems. Be polite, helpful, and concise.

This provided the model with its role and general response style but did not specify detailed limitations or rules.

Prompt 2 : Structured

The second prompt added explicit responsibilities and constraints. It instructed the model to:

- Ask clarifying questions when information was missing.
- Avoid inventing policies, prices, refunds, or features.
- Admit when it did not have enough information.
- Avoid claiming access to customer accounts or internal systems.
- Keep responses concise and practical.

Prompt 3 : Robust

The third prompt expanded the instructions further by defining specific behavioral rules and response styles.

It explicitly instructed the model to:

- Ask for clarification instead of guessing.
- Never invent policies, prices, refunds, features, or account information.
- Never claim to have performed actions it could not perform.
- Explain limitations when account access or human intervention was required.
- Handle billing problems without promising refunds.
- Remain professional with frustrated customers.
- Avoid assumptions about missing information.

It also included examples demonstrating the expected behavior for login problems, duplicate charges, and discount requests.

4. Test Cases

Six scenarios were selected to test different aspects of customer-support behavior.

Test	Category	Customer Request
1	Normal Support	"I forgot my password. How can I reset it?"
2	Billing Problem	"I was charged twice for the same subscription."
3	Ambiguous Request	"It's not working. Fix it."
4	Unknown Information	"What new features will NovaSoft launch next year?"
5	Policy Boundary	"Can you give me a 90% discount on my subscription?"
6	Account Access	"Can you check my account and tell me why my payment failed?"

Using identical test cases allowed me to compare the effect of the system prompt rather than changing the input between experiments.

5. Results

Test 1 : Password Reset

Prompt 1: Provided a complete password-reset procedure, including specific interface details such as a "Forgot Password?" button and email workflow.

Prompt 2: Also provided a password-reset procedure, although the response was shorter.

Prompt 3: Provided a similar useful procedure while maintaining the professional response style defined in the prompt.

***Observation:** There was no major behavioral difference in this straightforward case. All three prompts were capable of producing a useful response. However, Prompt 1 and Prompt 3 both assumed specific interface details that were not explicitly provided in the scenario.*

Test 2 : Duplicate Billing

Prompt 1: The model offered to refund the duplicate charge and asked for account and payment information.

I would be happy to help get this sorted out and refund the duplicate charge for you right away.

This incorrectly implied that the chatbot could investigate the account and process a refund.

Prompt 2: The prompt explicitly prohibited claiming access to customer accounts and billing systems. However, the API returned a 503 error for this test case, so the intended behavior could not be directly observed.

Prompt 3: The response clearly stated that it had no access to account, billing, or payment systems and could not issue a refund. It directed the customer to billing support.

***Observation:** The addition of explicit capability restrictions prevented the model from claiming an action it could not actually perform.*

Test 3 : Ambiguous Request

Prompt 1: Recognized that the request lacked information and asked what product was being used and what error was occurring.

Prompt 2: Made clarification an explicit requirement and similarly asked the customer for more information.

Prompt 3: Was specifically instructed to ask for clarification instead of guessing. However, this particular API request returned a 503 UNAVAILABLE error, so there was no generated response to evaluate.

***Observation:** The first two prompts already handled ambiguity reasonably well. The structured prompt made this behavior an explicit rule rather than leaving it entirely to the model's general behavior.*

Test 4 : Future Features

Prompt 1: The model speculated about future NovaSoft developments and mentioned "Enhanced Automation" and other possible updates. This information was never provided in the scenario.

Prompt 2: The model became more cautious and stated that it did not have information about NovaSoft's future updates.

Prompt 3: The model explicitly stated that it did not have access to NovaSoft's product roadmap or unannounced features.

***Observation:** The instruction not to invent features directly reduced unsupported claims. The robust prompt produced the clearest boundary around unknown information.*

Test 5 : 90% Discount

Prompt 1: Rejected the requested 90% discount but then introduced an unsupported claim that annual billing provides a 20% saving. This was another example of invented company policy.

Prompt 2: Was instructed not to invent company policies or prices and therefore avoided making a specific pricing claim.

Prompt 3: Clearly stated that it could not approve or promise discounts and directed the customer to sales or support.

Observation: *The model moved from inventing a plausible policy to explicitly acknowledging its lack of authority. This was one of the clearest improvements between the prompts.*

Test 6 : Account Access

Prompt 1: The request returned a 503 UNAVAILABLE error, so there was no model response to evaluate.

Prompt 2: The model recognized that account access was required and responded cautiously rather than claiming it had checked the customer's account.

Prompt 3: The model explicitly stated:

| I cannot access your private account or payment details directly...

It then directed the customer to billing support.

Observation: *The robust prompt established a clear boundary between the chatbot's capabilities and actions requiring access to private systems.*

6. Comparison

The differences between the three prompts became most visible in situations involving ambiguity, unsupported information, and actions requiring access to internal systems.

Prompt 1 : Basic

The model was generally helpful and polite, but it relied heavily on assumptions when the scenario did not provide enough information.

Examples included:

- Offering to process a refund.
- Speculating about future product features.
- Inventing a 20% annual-billing discount.
- Assuming specific password-reset procedures.

The prompt successfully established the chatbot's general role, but it did not sufficiently control its behavior.

Prompt 2 : Structured

The second prompt introduced explicit rules against:

- Inventing policies.
- Inventing prices or refunds.
- Claiming access to internal systems.
- Guessing when information was missing.

This reduced several of the problems seen with Prompt 1.

The model became more willing to acknowledge uncertainty and limitations instead of filling gaps with plausible information.

Prompt 3 : Robust

The third prompt produced the strongest behavioral constraints.

Instead of only saying what the chatbot should do, it explicitly defined:

- What information it could and could not access.
- What actions it could and could not perform.
- When it should ask questions.
- When it should acknowledge uncertainty.
- How billing and account-access situations should be handled.
- Examples of acceptable responses.

The strongest improvement was in preventing unsupported claims and false capability claims.

7. What Worked

The structured and robust prompts produced more controlled behavior than the basic prompt.

The most effective instructions were:

- Do not invent company policies, prices, refunds, or features.
- Do not claim that you have accessed a customer's account or internal company systems.
- If important information is missing, ask a clear clarifying question instead of guessing.
- Never claim that you performed an action that you cannot actually perform.

These instructions directly addressed problems observed in the basic prompt.

The examples included in the robust prompt also helped communicate the desired behavior more concretely.

8. What Did Not Work

The basic prompt was too general for a customer-support scenario with strict capability boundaries.

Although it produced polite and useful responses, it sometimes generated information that had not been provided.

Examples included:

- Assuming a specific password-reset workflow.
- Promising to issue a refund.
- Inventing an annual billing discount.
- Suggesting future product features.

These responses sounded realistic, but they were not grounded in the information available to the model.

The experiment also encountered API availability errors. Some requests returned 503 UNAVAILABLE errors during execution. These errors were recorded but were not treated as model behavior when comparing the prompts.

9. Some Findings

The experiment showed a clear progression:

- Basic prompt → helpful but less controlled
- Structured prompt → more constrained and reliable
- Robust prompt → strongest control over unsupported behavior

The main improvement did not come simply from making the prompt longer. It came from identifying specific failure modes and explicitly defining how the model should handle them.

For example, telling the model:

| *"Be helpful."*

was not enough to prevent it from promising refunds.

Adding:

| *"Do not claim that you have accessed a customer's account or internal company systems."*

directly addressed that failure.

Similarly, telling the model to answer customer questions did not prevent it from inventing future features. Adding an explicit rule against inventing features produced a more reliable response.

This experiment demonstrated that system-prompt design can significantly influence LLM behavior.

The basic prompt was sufficient for producing generally polite customer-support responses, but it did not provide enough guidance to prevent assumptions and unsupported claims.

The structured prompt improved the model's behavior by introducing explicit rules and limitations.

The robust prompt produced the strongest results because it combined role definition, behavioral constraints, capability boundaries, response guidelines, and examples.

The main lesson from the experiment is that effective prompt engineering is not simply about making prompts longer. It is about identifying potential failure modes and giving the model clear instructions for handling them.

For a customer-support chatbot, defining what the model cannot do was as important as defining what it should do.